SkyWay Airline Reservation System — Project Report

Oct 19, 2026 · @Rebee.AI

Overview and technologies

SkyWay is a desktop airline reservation system written in Java AWT, with MySQL (XAMPP) for storage and a QR-code payment page on the customer's phone. An admin manages flights and sees all bookings; a customer views flights, picks seats on a drawn aeroplane map, enters passengers, pays by scanning a QR code, and gets a PDF ticket by email.

Technology / library	Jar or tool	Used for
Java AWT	built into Java	All windows: Frame, Label, TextField, Button, Choice, List, TextArea, Canvas, Dialog
JDBC	mysql-connector-j 8.4.0	Connecting Java to MySQL, PreparedStatement queries, transactions
MySQL (XAMPP, phpMyAdmin)	XAMPP, port 3307	Database `airline_db`: admins, customers, flights, seats, bookings, payments
Apache PDFBox	pdfbox-app 3.0.3	Making the PDF ticket
Jakarta Mail + Angus Mail	jakarta.mail-api 2.1.3, angus-mail 2.0.3, activation jars	Sending the ticket by Gmail SMTP with the PDF attached
ZXing	core 3.5.3	Making the QR code of the payment link
com.sun.net.httpserver.HttpServer	built into Java	Small HTTP backend (port 8085) between the phone website and the Java app
React + Vite	Node.js, `payment-website/`	The payment web page opened from the QR code

Technology / library	Jar or tool	Used for
java.time	built into Java	Checking dates and times (LocalDateTime)

The Java code uses only `*` imports, the class style taught in class (create objects, design, event handling, add components, show the frame), GridLayout for layout, and no generics, lambdas or threads written in the project code.

Project structure: classes and their main methods

The project has 18 Java files in `src/` (compiled `.class` files go to `classes/`), jar files in `lib/`, the SQL script in `database/`, and the React website in `payment-website/`.

Class	Role	Main methods
`AirlineMain`	First window (Admin Login, Customer Login, Exit); starts the program	`main()` adds the Ctrl-shortcut listener, starts `PaymentServer`, opens the window; `findButton()` finds a button by its first letter
`DBConnection`	One place for the MySQL connection	`getConnection()`: `Class.forName` + `DriverManager.getConnection`
`LoginFrame`	Login for admin and customer (same window, type passed in)	`actionPerformed()`: checks username/email and password
`RegisterFrame`	New customer account	`actionPerformed()`: validates and inserts into `customers`
`AdminDashboard`	Admin menu: Manage Flights, View Bookings, Logout	`actionPerformed()`
`ManageFlightsFrame`	List, search, add, update, delete flights	`loadFlights()`, `actionPerformed()` (delete inside)
`FlightFormFrame`	Add a new flight or update one (same window)	`actionPerformed()` validates; `saveFlight()` saves flight + seats in one transaction
`ViewBookingsFrame`	Admin: all bookings, search by PNR	`loadBookings()`, `itemStateChanged()`

Class	Role	Main methods
`CustomerDashboard`	Customer menu: View Flights & Book, My Bookings, Logout	`actionPerformed()`
`AvailableFlightsFrame`	All flights with From/To filter and Available/Full/Departed status	`loadFlights()`, `actionPerformed()`
`SeatSelectionFrame`	Window around the seat map; total price; Continue	`loadSeats()`, `updateSelected()`, `actionPerformed()`
`SeatMap`	Draws the aeroplane and seats on a Canvas; handles seat clicks	`paint()`, `mousePressed()`, `seatX()`, `seatY()`, `findSelected()`
`PassengerDetailsFrame`	One row per passenger (seat, name, age, gender); books after payment	`actionPerformed()`, `confirmBooking()` (transaction + PNR), `bookingDone()`
`PaymentFrame`	Shows total fare and the QR code; waits for the phone	`newPayment()`, `paymentSuccess()`, `paymentFailed()`, `makeTicket()`, `closeWindow()`
`PaymentServer`	HTTP backend on port 8085 for the website	`start()`, `handleRequest()`, `getDetails()`, `paymentDone()`, `update()`, `getLaptopIp()`
`TicketService`	Ticket text, PDF, email, random codes	`getDetails()`, `makePdf()`, `sendEmail()`, `randomCode()`
`MyBookingsFrame`	Customer bookings, PNR search, cancel one ticket, resend ticket	`loadBookings()`, `actionPerformed()`, `cancelTicket()`, `itemStateChanged()`
`MessageBox`	Popup message and Yes/No box (AWT has no JOptionPane)	constructor shows it; `yes` tells if Yes was clicked

The program starts in `AirlineMain.main()`:

```
public static void main(String args[])
{
    // keyboard shortcut: Ctrl + first letter of a button clicks that button
(in every window)
    Toolkit.getDefaultToolkit().addAWTEventListener(new AWTEventListener() {
... }, AWTEvent.KEY_EVENT_MASK);

    PaymentServer.start(); // small HTTP server for the QR payment website
    new AirlineMain();
}
```

Database design

The database `airline_db` has 6 tables, created by `database/airline_db.sql` (imported in phpMyAdmin). All tables use InnoDB so transactions and foreign keys work.

Table	Key columns	Rules / constraints
`admins`	admin_id, username, password	username UNIQUE
`customers`	customer_id, name, email, phone, password	email UNIQUE
`flights`	flight_id (AUTO_INCREMENT), flight_no, airline, source, destination, departure_time, arrival_time, price, total_seats, available_seats	UNIQUE (flight_no, departure_time); CHECK available_seats between 0 and total_seats
`seats`	seat_id, flight_id, seat_no (1A, 1B ...), status AVAILABLE / BOOKED	UNIQUE (flight_id, seat_no); deleted with the flight (ON DELETE CASCADE)
`bookings`	booking_id, pnr, customer_id, flight_id, seat_id, passenger_name, passenger_age, passenger_gender, booking_date, status CONFIRMED / CANCELLED, ticket_price	one row per passenger; same PNR for all passengers of one booking; booking_date = CURRENT_TIMESTAMP
`payments`	payment_id, payment_ref, customer_id, flight_id, seats, passengers, amount, status PENDING / SUCCESS / FAILED, pnr, created_at	payment_ref UNIQUE

Passwords are stored as plain text (no hashing, as decided for this project). All user input goes into queries through `PreparedStatement` with `?`, which protects against SQL injection.

Login and registration

One `LoginFrame` serves both admin and customer: `new LoginFrame("Admin")` or `new LoginFrame("Customer")`. It reads the row by username (admin) or email (customer), then compares the password in Java so the check is case sensitive.

```
Connection con = DBConnection.getConnection();
String sql = "SELECT * FROM customers WHERE email = ?";
if (type.equals("Admin"))
    sql = "SELECT * FROM admins WHERE username = ?";
PreparedStatement ps = con.prepareStatement(sql);
ps.setString(1, user);
ResultSet rs = ps.executeQuery();

// password is checked in java so that it is case sensitive
if (!rs.next() || !rs.getString("password").equals(pass))
    new MessageBox(this, "Wrong login details");
else if (type.equals("Admin"))
    new AdminDashboard(user);
else
    new CustomerDashboard(rs.getInt("customer_id"), rs.getString("name"));
con.close();
```

`RegisterFrame` checks every field before saving: all fields filled, email format (regular expression), phone = 10 digits, password at least 4 characters, and both passwords equal. A repeated email is refused by the UNIQUE key on `customers.email`.

```
else if (!email.matches("[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\\.[A-Za-z]{2,}"))
    new MessageBox(this, "Please enter a valid email");
else if (!phone.matches("[0-9]{10}"))
    new MessageBox(this, "Phone number must be 10 digits");
```

After login the customer's `customer_id` is passed to every next window through the constructor (no session object is used); all bookings are saved in MySQL, so they are still there after logout.

Admin: manage flights and view bookings

The admin dashboard has 3 buttons: Manage Flights, View Bookings and Logout. Manage Flights shows every flight in an AWT `List` and has Search (flight number or ID), List All, Add Flight, Update Selected, Delete Selected and Back.

Search and list in one query. When the search box is empty, `? = ''` is true and every flight is shown:

```
PreparedStatement ps = con.prepareStatement("SELECT * FROM flights WHERE ? =
'' OR flight_no = ? OR flight_id = ? ORDER BY departure_time");
ps.setString(1, search);
ps.setString(2, search);
ps.setString(3, search);
```

Add and update use the same window (`FlightFormFrame` , flight id 0 = new flight). It checks: all fields filled, source not equal to destination, time format `yyyy-MM-dd HH:mm` (parsed with `LocalDateTime.parse`), arrival after departure, price above 0, seats 1 to 180, and a new flight's departure in the future. `saveFlight()` then runs one transaction:

1. Lock the flight row (`SELECT ... FOR UPDATE`) when updating.
2. Allow a change of total seats only if the flight has no CONFIRMED bookings.
3. INSERT or UPDATE the flight; for a new flight the AUTO_INCREMENT id is read with `getGeneratedKeys()` .
4. Create the seats 1A, 1B ... 1F, 2A ... (6 per row) and `commit()` ; on any error `rollback()` .

```
PreparedStatement ps3 = con.prepareStatement("INSERT INTO seats (flight_id,
seat_no) VALUES (?, ?)");
for (int i = 0; makeSeats && i < seats; i++)
{
    ps3.setInt(1, id);
    ps3.setString(2, (i / 6 + 1) + "" + "ABCDEF".charAt(i % 6));
    ps3.executeUpdate();
}
```

Delete is blocked while a flight has active bookings, using one safe query:

```
DELETE FROM flights WHERE flight_id = ? AND NOT EXISTS
    (SELECT * FROM bookings WHERE flight_id = ? AND status = 'CONFIRMED')
```

View Bookings (`ViewBookingsFrame`) lists every ticket of every customer (one row per passenger) with a PNR search; clicking a row shows the full ticket from `TicketService.getDetails(pnr)` .

Customer: available flights

The customer sees every flight the admin added, each with a status of Available, Full or Departed; only Available flights can be booked. Two `Choice` boxes (From, To) are filled with city names from the database, with "All" first.

```
ResultSet rs = stmt.executeQuery("SELECT source AS city FROM flights UNION
SELECT destination FROM flights ORDER BY city");
while (rs.next())
{
    fromChoice.add(rs.getString("city"));
    toChoice.add(rs.getString("city"));
}
```

One query handles both "all cities" and a filter, and MySQL's `NOW()` marks flights whose time is over:

```
PreparedStatement ps = con.prepareStatement("SELECT *, departure_time < NOW()
AS departed FROM flights "
        + "WHERE (? = 'All' OR source = ?) AND (? = 'All' OR destination = ?)
ORDER BY departure_time");
...
String st = "Available";
if (rs.getInt("departed") == 1)
    st = "Departed";
else if (rs.getInt("available_seats") == 0)
    st = "Full";
```

Each row is added to the List with `String.format("%-4d %-7s %-10s ...")` and a Monospaced font, so the columns line up under the heading label. The flight id and status of each row are kept in two arrays (`int[] flightIds` , `String[] status`) so the selected row's flight can be opened in Select Flight.

Seat selection: the aeroplane seat map

`SeatMap` extends `Canvas` and draws the plane itself in `paint()` : a rounded nose with cockpit windows and the word FRONT, the body, a rounded tail, letters A to F, row numbers in the aisle, and one rounded box per seat. White = available, green = selected, grey with X = booked. A customer can select up to 6 seats; clicking a selected seat removes it.

Where each seat is drawn comes from its index `i` (seats are read ordered by seat_id: 1A, 1B ... 1F, 2A ...):

```
// x position of seat i (i / 6 = row number, rows go from left to right)
int seatX(int i)
{
    return startX + (i / 6) * (seat + 8);
}

// y position of seat i (i % 6 = A to F, D E F come after the aisle)
int seatY(int i)
{
    int y = top + 25 + (i % 6) * (seat + gap);
    if (i % 6 >= 3)
        y = y + aisle - gap;
    return y;
}
```

Drawing with Graphics2D: `fillOval` for the nose and tail, `fillRect` for the body, `fillRoundRect` / `drawRoundRect` for seats, `drawString` for letters. Antialiasing makes the curves smooth.

Fits any window: before drawing, a scale is worked out so the whole plane fits, then `g2.scale()` shrinks or grows everything:

```
scale = Math.min(1, Math.min((getWidth() - 20.0) / (right + 20), (getHeight()
- 10.0) / (bottom + 20)));
g2.translate(offX, offY);
g2.scale(scale, scale);
```

No flicker (double buffering): everything is drawn on an `Image` first, then the finished image is shown with `g.drawImage(buffer, 0, 0, this)`; `update()` is overridden to call `paint()` directly.

Clicking a seat: `SeatMap` implements `MouseListener`. In `mousePressed()` the mouse position is changed back to drawing size, each seat is tested, and the `selected` array (`int[6]`) is updated:

```
int x = (int) ((e.getX() - offX) / scale);
int y = (int) ((e.getY() - offY) / scale);
for (int i = 0; i < seatNos.length; i++)
{
    boolean inside = x >= seatX(i) && x <= seatX(i) + seat && y >= seatY(i)
&& y <= seatY(i) + seat;
    ...
}
repaint();
frame.updateSelected(); // shows "Selected seats: 1A 1B | Total: Rs. 9000.0"
```

Seats were drawn instead of using AWT Buttons because macOS ignores AWT button colours.

Multiple passengers, safe booking and PNR

One booking can hold up to 6 passengers (family and friends), all under one PNR. `PassengerDetailsFrame` shows one row per selected seat (seat, name, age, gender) in a `GridLayout(count + 3, 4)`. The rows are added in a loop and read back with `getComponent()`, because each row has exactly 4 parts:

```
// first 8 components are title and headings, then 4 for every passenger
TextField nameTf = (TextField) getComponent(8 + i * 4 + 1);
TextField ageTf = (TextField) getComponent(8 + i * 4 + 2);
Choice gender = (Choice) getComponent(8 + i * 4 + 3);
```

Seats are booked only after the payment is done. `confirmBooking()` does it in one JDBC transaction, so a seat can never be half booked or booked twice:

1. `con.setAutoCommit(false)` starts the transaction.
2. Lock the flight row and check it has not departed: `SELECT price FROM flights WHERE flight_id = ? AND departure_time > NOW() FOR UPDATE`.
3. Mark each seat BOOKED only if it is still AVAILABLE; if 0 rows change, someone else just took it and everything is undone.
4. Decrease `available_seats` by the number of passengers.
5. Make a PNR that is not used before, insert one bookings row per passenger with that PNR, then `con.commit()`. Any error → `con.rollback()`.

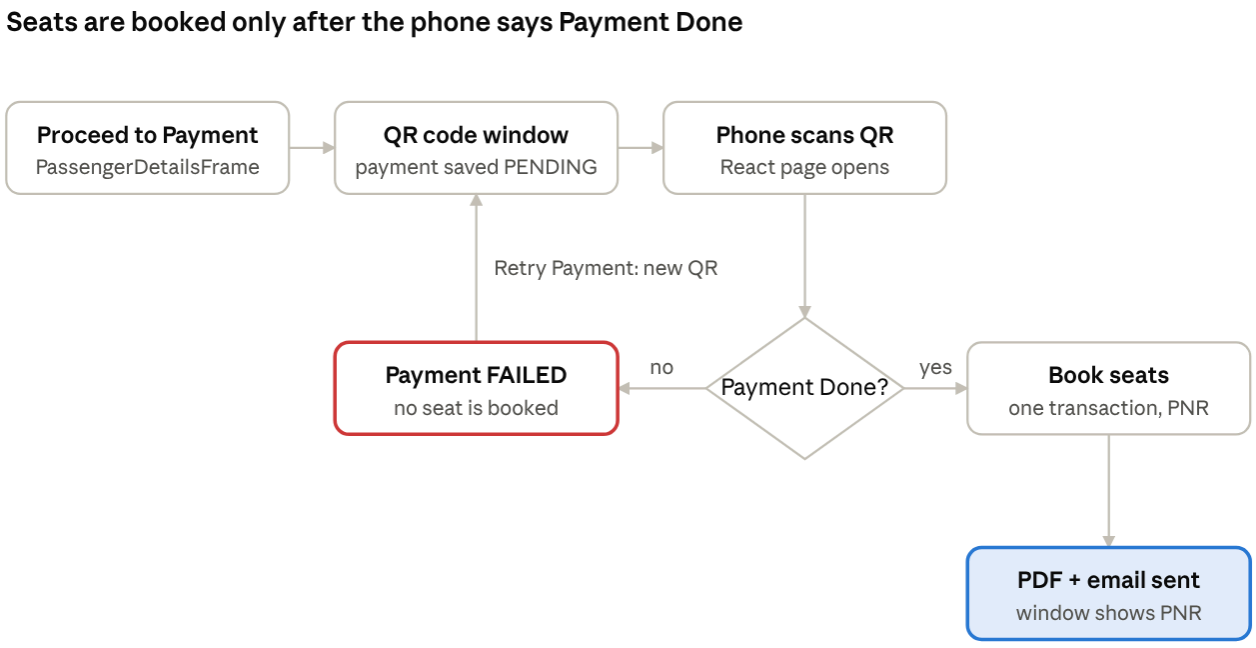
```
PreparedStatement ps2 = con.prepareStatement("UPDATE seats SET status =
'BOOKED' WHERE seat_id = ? AND status = 'AVAILABLE'");
for (int i = 0; i < count; i++)
{
    ps2.setInt(1, seatIds[i]);
    if (ps2.executeUpdate() != 1)
        throw new Exception("Sorry, seat " + seatNos[i] + " was just booked
by someone else");
}
```

PNR generation (`TicketService.randomCode`): `Random.nextInt()` picks 6 characters from 28 allowed ones; look-alike characters B G I O S Z are left out so a PNR is never misread. A `while` loop makes a new code if it already exists in `bookings`.

```
public static String randomCode(int length)
{
    String chars = "ACDEFHJKLMNPQRTUVWXY23456789"; // no B G I O S Z, they
look like 8 6 1 0 5 2
    Random r = new Random();
    String code = "";
    for (int i = 0; i < length; i++)
        code = code + chars.charAt(r.nextInt(chars.length()));
    return code;
}
```

The same method makes payment references: `"PAY" + TicketService.randomCode(6)`.

QR code payment



QR payment flow · Payment Done books the seats, Payment Failed books nothing

The payment is a simulation: no money moves and no bank details are asked. The phone and the laptop must be on the same network.

1. **PaymentFrame** saves a row in `payments` with status PENDING and a reference like `PAY7FLYPQ`, then draws a QR code of the website link with ZXing.
2. The phone scans it and opens the React page `http://<laptop IP>:5173/?ref=PAY7FLYPQ`, which asks the Java server for the flight, seats and amount.
3. **Payment Done** → `PaymentServer` changes PENDING to SUCCESS, calls `PaymentFrame.paymentSuccess()` → `confirmBooking()` books the seats and gets the PNR; the PDF and email follow, and the laptop window updates by itself.

4. **Payment Failed** → status FAILED, nothing is booked; **Retry Payment** on the laptop makes a new payment and a new QR code. Closing the window also marks the payment FAILED.

QR code with ZXing, drawn square by square on a Canvas:

```
String url = "http://" + PaymentServer.getLaptopIp() + ":5173/?ref=" + ref;
qr = new QRCodeWriter().encode(url, BarcodeFormat.QR_CODE, 1, 1);
...
for (int x = 0; x < qr.getWidth(); x++)
    for (int y = 0; y < qr.getHeight(); y++)
        if (qr.get(x, y))
            g.fillRect(x0 + x * box, y0 + y * box, box, box);
```

The small HTTP backend uses Java's built-in `HttpServer` (no extra jar) on port 8085 with 3 addresses: `/api/payment/details`, `/success`, `/fail`. Java's `HttpServer` handles each request on its own background thread; the project code itself creates no threads.

```
HttpServer server = HttpServer.create(new InetSocketAddress(8085), 0);
server.createContext("/api/payment", new HttpHandler()
{
    public void handle(HttpExchange ex) throws IOException
    {
        handleRequest(ex);
    }
});
server.start();
```

Clicking twice cannot book twice: the status changes only while it is still PENDING.

```
if (update("UPDATE payments SET status = 'SUCCESS' WHERE payment_ref = ? AND
status = 'PENDING'", ref) != 1)
    return message(false, "This payment is already completed or failed.");
```

Laptop IP for the QR link is found by connecting a UDP socket (nothing is sent): `ds.connect(InetAddress.getByName("8.8.8.8"), 80);` `ds.getLocalAddress().getHostAddress()`.

The React page (`payment-website/src/App.jsx`, run with `npm run dev`) shows the booking reference, From → To, flight, departure, passengers, seats and amount, plus the Payment Done and Payment Failed buttons, and calls the server with `fetch(API + '/success?ref=' + ref, { method: 'POST' })`.

PDF ticket and email

All three parts live in `TicketService`: `getDetails()` builds the ticket text, `makePdf()` writes it into a PDF, and `sendEmail()` mails the PDF to the customer.

Ticket text (`getDetails`): one query joins bookings, seats, customers and flights; each row is one passenger. Flight details come from the first row, each passenger becomes one line with `String.format`, and Total Fare adds only CONFIRMED tickets.

```
list = list + String.format("%-20s %-12s %-6s %-10s %.2f\n",
rs.getString("passenger_name"),
        rs.getInt("passenger_age") + " / " +
rs.getString("passenger_gender"), rs.getString("seat_no"),
        rs.getString("status"), rs.getDouble("ticket_price"));
if (rs.getString("status").equals("CONFIRMED"))
    total = total + rs.getDouble("ticket_price"); // cancelled tickets are
not counted
```

PDF with Apache PDFBox (`makePdf`): a Helvetica Bold title, then every line of the ticket text in Courier (all letters the same width, so the table columns line up), a box around it, saved as `tickets/ticket_<PNR>.pdf`.

```
PDDocument doc = new PDDocument();
PDPage page = new PDPage();
doc.addPage(page);
PDPageContentStream cs = new PDPageContentStream(doc, page);
cs.beginText();
cs.setFont(new PDType1Font(Standard14Fonts.FontName.COURIER), 11);
cs.setLeading(16);
cs.newLineAtOffset(60, 650);
for (int i = 0; i < lines.length; i++)
{
    cs.showText(lines[i]);
    cs.newLine();
}
cs.endText();
...
doc.save(path);
```

Email with Jakarta Mail (`sendEmail`) through Gmail SMTP:

1. Read the Gmail address and App Password from `config.properties` (kept out of the code and out of Git).
2. Find the customer's name and email from MySQL using the PNR.

3. Connect to `smtp.gmail.com`, port 587, with STARTTLS and log in with an `Authenticator`.
4. Build a `MimeMessage`: subject `Your E-Ticket - PNR <PNR>`, a text part with the ticket details, and the PDF as an attachment, joined with `MimeMultipart`.
5. `Transport.send(msg)`.

```
props.put("mail.smtp.auth", "true");
props.put("mail.smtp.starttls.enable", "true");
props.put("mail.smtp.host", "smtp.gmail.com");
props.put("mail.smtp.port", "587");
...
MimeBodyPart filePart = new MimeBodyPart();
filePart.attachFile(new File(pdfPath));
Multipart mp = new MimeMultipart();
mp.addBodyPart(textPart);
mp.addBodyPart(filePart);
msg.setContent(mp);
Transport.send(msg);
```

If the email fails (no internet, wrong App Password) the booking and the PDF are still saved, and the customer can resend from My Bookings with Get Ticket (PDF + Email).

My Bookings and cancellation

`MyBookingsFrame` lists only the logged-in customer's tickets, one row per passenger, with a PNR search box and the buttons Search, List All, Cancel Ticket, Get Ticket (PDF + Email) and Back. Clicking a row (`ItemListener.itemStateChanged`) shows the full ticket in the TextArea below.

```
PreparedStatement ps = con.prepareStatement("SELECT b.*, s.seat_no,
f.flight_no, f.source, f.destination, f.departure_time FROM bookings b "
        + "JOIN seats s ON b.seat_id = s.seat_id JOIN flights f ON
b.flight_id = f.flight_id "
        + "WHERE b.customer_id = ? AND (? = '' OR b.pnr = ?) ORDER BY
b.booking_date DESC, b.pnr, s.seat_id");
```

Cancel Ticket cancels one passenger only; the others on the same PNR stay booked. After a Yes/No `MessageBox`, `cancelTicket()` runs one transaction:

1. Lock the booking row with `FOR UPDATE` and check it belongs to this customer.
2. Refuse if it is already CANCELLED or the flight has departed (`departure_time < NOW()`).

3. Set the booking to CANCELLED, set the seat back to AVAILABLE, add 1 to `available_seats`.
4. `commit()`; any error → `rollback()`.

Because the row is locked and its status is checked first, a ticket cannot be cancelled twice and the seat count is increased only once. The ticket total then counts only the passengers still CONFIRMED.

Keyboard shortcuts: Ctrl + first letter

In every window, Ctrl + the first letter of a button clicks that button (Ctrl+L = Login, Ctrl+S = Search, Ctrl+B = Back). It is written once in `AirlineMain.main()` and works everywhere; on a Mac the Control key is used.

1. `Toolkit.getDefaultToolkit().addAWTEventListener(...)` receives every key press of the whole program, not only of the focused box.
2. It reacts only to KEY_PRESSED with Ctrl held and a letter A to Z.
3. A `while` loop with `getParent()` climbs from the pressed component to its Window, so only buttons of that window can be clicked.
4. `findButton()` searches the window and, by recursion, every container inside it for an enabled button whose label starts with that letter.
5. `dispatchEvent(new ActionEvent(...))` sends the button the same event as a mouse click, so its normal `actionPerformed()` runs.

```
Toolkit.getDefaultToolkit().addAWTEventListener(new AWTEventListener()
{
    public void eventDispatched(AWTEvent ev)
    {
        KeyEvent ke = (KeyEvent) ev;
        int key = ke.getKeyCode();
        if (ke.getID() != KeyEvent.KEY_PRESSED || !ke.isControlDown() || key
< 'A' || key > 'Z')
            return;

        // finding the window in which the key was pressed
        Component c = ke.getComponent();
        while (c != null && !(c instanceof Window))
            c = c.getParent();

        Button btn = null;
        if (c != null)
            btn = findButton((Container) c, key);
        if (btn != null)
```

```
        {
            ke.consume();
            btn.dispatchEvent(new ActionEvent(btn,
ActionEvent.ACTION_PERFORMED, btn.getLabel()));
        }
    }
}, AWTEvent.KEY_EVENT_MASK);
```

```
static Button findButton(Container box, int key)
{
    Component[] list = box.getComponents();
    for (int i = 0; i < list.length; i++)
    {
        Button b = null;
        if (list[i] instanceof Button)
            b = (Button) list[i];
        else if (list[i] instanceof Container)
            b = findButton((Container) list[i], key);

        if (b != null && b.isEnabled() &&
b.getLabel().trim().toUpperCase().charAt(0) == key)
            return b;
    }
    return null;
}
```

Every button in a window starts with a different letter (for example Show All was renamed List All), so each shortcut clicks exactly one button.

Real date and time

The project uses the laptop's real current date and time, from MySQL's `NOW()` and Java's `LocalDateTime.now()`. A flight that departed at 14:30 shows Departed at 14:31.

Where	What it checks	Code
Available Flights	Marks flights whose time is over as Departed; they cannot be booked	`departure_time < NOW() AS departed`
Booking after payment	Checks again that the flight has not departed before saving	`departure_time > NOW() FOR UPDATE`

Where	What it checks	Code
Cancel Ticket	A ticket cannot be cancelled after the flight has departed	`departure_time < NOW() AS departed`
Add Flight	A new flight's departure must be in the future	`dep.isBefore(LocalDateTime.now())`
Booking date	Saved automatically at the moment of booking	`booking_date DATETIME DEFAULT CURRENT_TIMESTAMP`
Payment record	The time each payment was created	`created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP`

Dates typed by the admin are checked with `LocalDateTime.parse(text.replace(" ", "T"))`, so a wrong format such as `2026-2-3 8:00` shows an error message instead of saving.

Other techniques used in the code

One window style everywhere. Each window is a class that `extends Frame` and builds itself in the constructor in the same order: setting the layout, creating objects with `new`, design (colours and fonts), event handling, adding components, closing the window, creating the frame (`setTitle`, `setSize`, `setLocationRelativeTo(null)`, `setVisible(true)`). Clicks are handled in `actionPerformed()` starting with `Button btn = (Button) e.getSource();`.

Only GridLayout. Simple windows put components straight on the Frame, for example `setLayout(new GridLayout(4, 2, 0, 10))` for Login. The bigger windows (both flight lists, both booking lists, seat selection, payment) split the frame into rows with GridLayout and use small GridLayout panels for rows with several buttons.

Listeners instead of adapter classes. Each window `implements WindowListener` and writes all 7 methods; only `windowClosing()` has code. `SeatMap` `implements MouseListener` the same way.

```
public void windowClosing(WindowEvent we)
{
    dispose();
}
public void windowOpened(WindowEvent we) {}
public void windowClosed(WindowEvent we) {}
public void windowIconified(WindowEvent we) {}
public void windowDeiconified(WindowEvent we) {}
public void windowActivated(WindowEvent we) {}
public void windowDeactivated(WindowEvent we) {}
```

MessageBox. AWT has no JOptionPane, so `MessageBox extends Dialog` is a modal popup: `new MessageBox(this, "text")` for a message, `new MessageBox(this, "Delete?", true).yes` for a Yes/No question.

Simple data structures. Plain arrays with a counter (`int[] flightIds = new int[500]; int total = 0;`) instead of ArrayList or generics.

Safe database work. `PreparedStatement` with `?` for all input; transactions with `setAutoCommit(false)`, `commit()` and `rollback()` for adding flights, booking and cancelling; `SELECT ... FOR UPDATE` row locks so two customers cannot book the same seat.

How to run the project

One-time setup:

1. Start MySQL in XAMPP, open phpMyAdmin, and import `database/airline_db.sql` (Import tab).
2. Check the port in `DBConnection.java` (this laptop's XAMPP uses 3307; most use 3306).
3. For email, put a Gmail address and a Gmail App Password in `config.properties` (`mail.username`, `mail.password`).
4. In `payment-website/`, run `npm install` once (needs Node.js).

Every time (two Terminals, MySQL running, phone on the same network as the laptop):

```
# Terminal 1 - payment website
cd "payment-website"
npm run dev
```

```
# Terminal 2 - Java app (compiles, then runs AirlineMain; also starts the
payment server)
javac -d classes -cp "lib/*" src/*.java
java -cp "classes:lib/*" AirlineMain
```

`./run.sh` does both Java steps at once (`run.bat` on Windows, where the classpath uses `;`). Sample logins: admin / admin123, and rahul@gmail.com / 1234.